

TruthTable++: Verifiable Query Engine with complete TPC-H support

Alireza Shirzad
Upenn

Bharath Namboothiry
Upenn

Ryan Marcus
Upenn

Pratyush Mishra
Upenn

September 3, 2026

Abstract

In the paper we present something cool

Contents

1	Introduction	1
2	Preliminaries	2
3	Arithmetization	4
3.1	General Approach	4
3.2	String Arithmetization	4
4	String Toolbox	7
4.1	Broadcast Check	7
4.2	Consistency-Checking protocols	7
4.3	Length Filtering	8
5	PIOPs for Wild-Card Pattern Matching	10
5.1	Multi-Character	10
6	Pre-Filtering Optimization	15
6.1	Pre-Filtering via Fingerprinting	15
6.2	A PIOP for Pre-Filtering	15
6.3	Bitwise AND	17
7	Implementation and Evaluation	18
7.1	Implementation	18
7.2	Evaluation	18
	Acknowledgments	19
A	Extended Preliminaries	20
A.1	s-Contiguous Polynomial Identity	20
A.2	Deferred Low-Level PIOPs	20
B	Deferred PIOPs	21
B.1	NoZero Check	21
B.2	Booleanity Check	22
B.3	No-Duplicate Check	22
B.4	Limb Reconstruction Check	22
B.5	Lookup Check	23
B.6	Range Check	23
B.7	Sign Check	24
B.8	Keyed Sumcheck	25
B.9	Multiset Equality Check	26
B.10	Rotation Check	26
	References	28

1 Introduction

[Ali: no other prior works support string operations] [Ali: some statistics: X percent of the queries out there have string predicates, Y percent of the TPCB queries have string predicates. A verifiable database without string support loses all these applications, something like that]

2 Preliminaries

We introduce notation and cryptographic background required to understand TruthTable++’s protocols.

Boolean hypercube. The μ -dimensional Boolean hypercube $\mathcal{H}_\mu$ is the set $\{0, 1\}^\mu$. It has size $n = 2^\mu$. We will commonly interpret elements of $\mathcal{H}_\mu$ as integers in the range $[0, n - 1]$.

Polynomials. We write $p \in \mathbb{F}^{\leq d}[X_1, \dots, X_\mu]$ (or, concisely, $p \in \mathbb{F}^{\leq d}[\mathbf{X}]$) to denote a polynomial in μ variables over the field $\mathbb{F}$, where each variable has degree at most d .

Multilinear extensions. Let $v \in \mathbb{F}^N$ be a vector of field elements with $N = 2^\mu$ for some $\mu \in \mathbb{N}$. The *multilinear extension* $\tilde{v}$ of v is the unique multilinear polynomial whose evaluations at the i -th point in the μ -dimensional boolean hypercube equal the i -th element of v . We now recall some useful multivariate polynomials.

- The *index polynomial* $\mathcal{I}_\mu(X) := \sum_{i=1}^\mu 2^{i-1} X_i$ maps the i -th point in $\mathcal{H}_\mu$ to the value $i - 1$.
- The *equality polynomial* $\text{Eq}(X, Y) := \prod_{i=1}^\mu ((1 - X_i)(1 - Y_i) + X_i Y_i)$ satisfies the property that, for $x, y \in \mathcal{H}_\mu$, $\text{Eq}(x, y)$ equals 1 if $x = y$, and equals 0 otherwise.
- The *s-contiguous polynomial* $\mathcal{C}_{\mu,s}(X) := \sum_{i=1}^\mu s_i (1 - X_i) \prod_{j=1}^{i-1} (s_j X_j + (1 - s_j)(1 - X_j))$ is 1 for the first s points in $\mathcal{H}_\mu$ and 0 elsewhere. Here, s_i is the i -th most significant bit of s . We provide a derivation of this polynomial in Section A.1.

Polynomial oracles. Algorithms with *oracle* access to a polynomial p (denoted $\llbracket p \rrbracket$) can query p at any point $x \in \mathbb{F}^\mu$ to obtain the evaluation $p(x)$.

Polynomial commitments. A polynomial commitment scheme is a cryptographic primitive that allows one to generate a succinct commitment cm_p to a polynomial p , and later reveal its evaluation at specific points, along with a proof that the evaluation is correct [KZG10].

SNARKs via polynomials. TruthTable++ follows the now-standard methodology for constructing SNARKs by combining *Polynomial Interactive Oracle Proofs* (PIOPs) with *Polynomial Commitment* (PC) schemes [BFS20; CHMMVW20]. A PIOP is an idealized interactive proof between a prover $\mathbf{P}$ and a verifier $\mathbf{V}$, in which the prover sends polynomial oracles and the verifier queries these oracles before deciding acceptance.

All protocol design in TruthTable++ occurs at the PIOP layer. We build PIOPs that prove correct execution of relational operators over database tables, and build these protocols compositionally using polynomial interactive oracle *reductions* [BCGGRS19; BMNW25; BMMS25], which interactively reduce complex claims to simpler algebraic subclaims. As shown in Section B, each operator-level protocol ultimately reduces to a small collection of low-level algebraic claims, which are proven using standard PIOPs from the literature. For completeness, we briefly recall the three protocols frequently used in TruthTable++, deferring formal definitions to Section A.2.

- **Sumcheck** proves claims of the form $\sum_{x \in \mathcal{H}_\mu} p(x) = s$ for a polynomial p and field element s .
- **Zerocheck** proves that a polynomial p vanishes on the Boolean hypercube, i.e., $p(x) = 0$ for all $x \in \mathcal{H}_\mu$.
- **Rational Sumcheck** proves claims of the form $\sum_{x \in \mathcal{H}_\mu} \frac{p(x)}{q(x)} = s$ for polynomials p, q , under the promise that $q(x) \neq 0$ for all $x \in \mathcal{H}_\mu$.

To obtain a SNARK, we compile these PIOPs using a polynomial commitment scheme [CHMMVW20; BFS20]. Intuitively, this compilation emulates the polynomial oracles in the PIOP model: each polynomial oracle sent by the prover is replaced by a *polynomial commitment*, and each oracle query is replaced by a proof that the committed polynomial evaluates to the claimed value at the verifier’s chosen point [CHMMVW20; GWC19]. Since this compilation applies generically, we treat it as a black box and focus exclusively on

designing efficient PIOPs for relational operators; the resulting SNARK inherits its efficiency and soundness directly from the underlying PIOPs and commitment scheme.

3 Arithmetization

We now describe how we represent columns and tables as polynomials for use with the PIOPs of the following sections. We briefly review the general arithmetization of tables and columns in Section 3.1, which we adopt from TruthTable [NSSMM26], and then turn in Section 3.2 to the arithmetization of strings—the focus of our paper and absent from TruthTable.

3.1 General Approach

Throughout, tables and columns are encoded as multilinear polynomials over the Boolean hypercube $\mathcal{H}_\mu$, with arithmetic in a finite field $\mathbb{F}$ of large prime characteristic p .

Notation: arithmetized objects. A concrete table or column is written plainly as T or c , and its *arithmetized* polynomial form carries a hat, $\hat{T}$ or $\hat{c}$. A concrete object is indexed by integers in square brackets, as in $c[i]$ or $T.\text{key}[i]$ for $i \in [|T|]$, whereas its arithmetization is evaluated at hypercube points in parentheses, as in $\hat{T}.\text{key}(x)$ for $x \in \mathcal{H}_\mu$.

Representing table data as field elements. We map each column into the prime field $\mathbb{F}$ according to its declared type. For all non-string types—integers, booleans, durations, intervals, dates, and timestamps—we follow TruthTable [NSSMM26], to which we refer the reader for the encodings. The encoding of strings as field elements is the subject of Section 3.2.

Columns. A column c of n entries lives in $\mathcal{H}_\mu$ for any $\mu \geq \lceil \log_2 n \rceil$, and its arithmetization splits into two multilinear polynomials, $\hat{c} := (\hat{c}.a, \hat{c}.d)$. The *activator* $\hat{c}.a: \mathbb{F}^\mu \rightarrow \{0, 1\}$ flags the n *active* indices—those carrying real rows rather than padding—while the *data* polynomial $\hat{c}.d: \mathbb{F}^\mu \rightarrow \mathbb{F}$ holds the entries at those indices. Where the column is clear from context, we abbreviate the two to a and d .

Tables. A k -column table T with n rows arithmetizes to $\hat{T} := (\hat{T}.a, [\hat{T}.d_1, \dots, \hat{T}.d_k])$: a single activator $\hat{T}.a$, active at n points of $\mathcal{H}_\mu$, shared across all k data polynomials. Each active point x —one with $\hat{T}.a(x) = 1$ —then yields a row $(\hat{T}.d_1(x), \dots, \hat{T}.d_k(x))$ of T . A column is just the $k = 1$ case, and we reuse the $\hat{T}.a, \hat{T}.d_i$ notation for tables.

Active indices and layouts. The *active set* of $\hat{c}$ (or $\hat{T}$) gathers its active points, $A(\hat{c}) := \{x \in \mathcal{H}_\mu \mid \hat{c}.a(x) = 1\}$, and for a value $k \in \mathbb{F}$ we single out those carrying it as $A_k(\hat{c}) := \{x \in A(\hat{c}) \mid \hat{c}.d(x) = k\}$.

We use one of two layouts for the active set. In the default *contiguous* layout, $A(\hat{T}) = \{0, \dots, n-1\}$, so the activator is the closed-form n -contiguous polynomial $C_{\mu,n}$ (Section 2) that the prover need neither commit to nor prove boolean. In the *sparse* layout the active indices are scattered arbitrarily, and the prover must commit to the activator and prove its booleanity.

Arithmetized relations. Every operator PIOP proves membership in a *table language* $\mathcal{L}_{\text{op}} = \{(x, \{\hat{I}\}, \{\hat{O}\})\}$, the input and output tables linked by the operator op . Its *arithmetized* counterpart replaces each concrete table with its polynomial form, $\hat{\mathcal{L}}_{\text{op}} := \{(\mathbb{x} = x, \mathbb{y} = (\{\hat{I}\}, \hat{O})) \mid ((x, \{\hat{I}\}, \hat{O}), \perp) \in \mathcal{L}_{\text{op}}\}$, where each $\hat{T} \in \{\hat{I}\} \cup \{\hat{O}\}$ is the arithmetization of the corresponding concrete table T .

3.2 String Arithmetization

We now describe how we arithmetize a string-typed column. This string arithmetization is a contribution of our work, and is designed so that the PIOP protocols of the following sections can operate directly on the contents of the strings.

We start with the approach of TruthTable [NSSMM26] which arithmetizes a string-typed column as a single hash column (a, h) , which acts as a black box handle to the strings and allows no white-box operation

on the strings. We denote this as our main column. However, to enable white-box operations on the string contents, we introduce *auxiliary columns*. The auxiliary columns are augmented to the hash column and come in two types with two different lengths:

Type 1: String-level auxiliary columns. These are columns of length N —the length of the hash column, with one entry per string:

- *Length column* l , with $l[i]$ being the length of the i -th string in c .

Type 2: Character-level auxiliary columns. These are columns of length $M := \sum_{s \in c} |s|$, the total number of characters in the column:

- *Character column* char contains the individual characters of all strings in c concatenated to each other.
- *Origin index column* orig-ind , with $\text{orig-ind}[j]$ being the index of the string in c to which the j -th character of char belongs.
- *Internal index column* int-ind , with $\text{int-ind}[j]$ being the position of the j -th character of char within its own string, counting from 0. Equivalently, int-ind resets to 0 at every string boundary and increments by one across the characters of each string.
- *Character activator column* char-act , with $\text{char-act}[j]$ being 1 if the j -th character of char is active, and 0 otherwise.
- *Boundary marker column* bnd , with $\text{bnd}[j]$ being 1 if the j -th character of char is the first character of a string (a string boundary), and 0 otherwise. **[Ali: This can be potentially removed from the arithmetization and be sent from the prover as $\text{int-ind}==0$ indicator. However, it comes with online prover cost.]**

Fig. 1 illustrates this arithmetization on a sample four-row string-typed column.

		apple					orange					cherry					banana									
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	31
s	a h																									
apple	1 h_0	1																								
orange	1 h_1		1																							
cherry	0 h_2			1																						
banana	1 h_3				1																					

	l	5	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6
--	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

char	a	p	p	l	e	o	r	a	n	g	e	c	h	e	r	r	y	b	a	n	a	n	a	0	...	0
orig-ind	0	0	0	0	0	1	1	1	1	1	1	2	2	2	2	2	2	3	3	3	3	3	3	0	...	0
int-ind	0	1	2	3	4	0	1	2	3	4	5	0	1	2	3	4	5	0	1	2	3	4	5	0	...	0
char-act	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	1	1	1	1	1	1	0	...	0
bnd	1	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	...	0

main columns
String-level Auxiliary column
Character-level Auxiliary columns

Figure 1: String arithmetization of a four-row column $c = (\text{apple}, \text{orange}, \text{cherry}, \text{banana})$ with cherry inactive.

Dropping Auxiliary Columns. The final result check needs only the hash column for the opening process, so the auxiliary columns need not be carried all the way through the plan. Deciding *how long* to keep them matters for efficiency: every extra column makes the downstream operations more expensive, especially the character-level columns, which can be far larger than the original string column. The best place to drop them is the earliest point past which the plan performs no further white-box string operation.

This motivates the planner to push string-filtering operations as far down the plan as possible, so the auxiliary columns can be released early. As Fig. 2 illustrates, such push-down is possible in some plans but not in others. Plan (a) applies a NOT LIKE predicate to `o_comment` above a LEFT JOIN: since the join’s right input is nullable, filtering the base `orders` table is not equivalent to filtering after the join, so the predicate is stuck above the join and the string auxiliary columns must be carried through it. Plans (b) and (c), by contrast, apply a LIKE predicate to the base column `p_name` across an inner JOIN, where the filter commutes

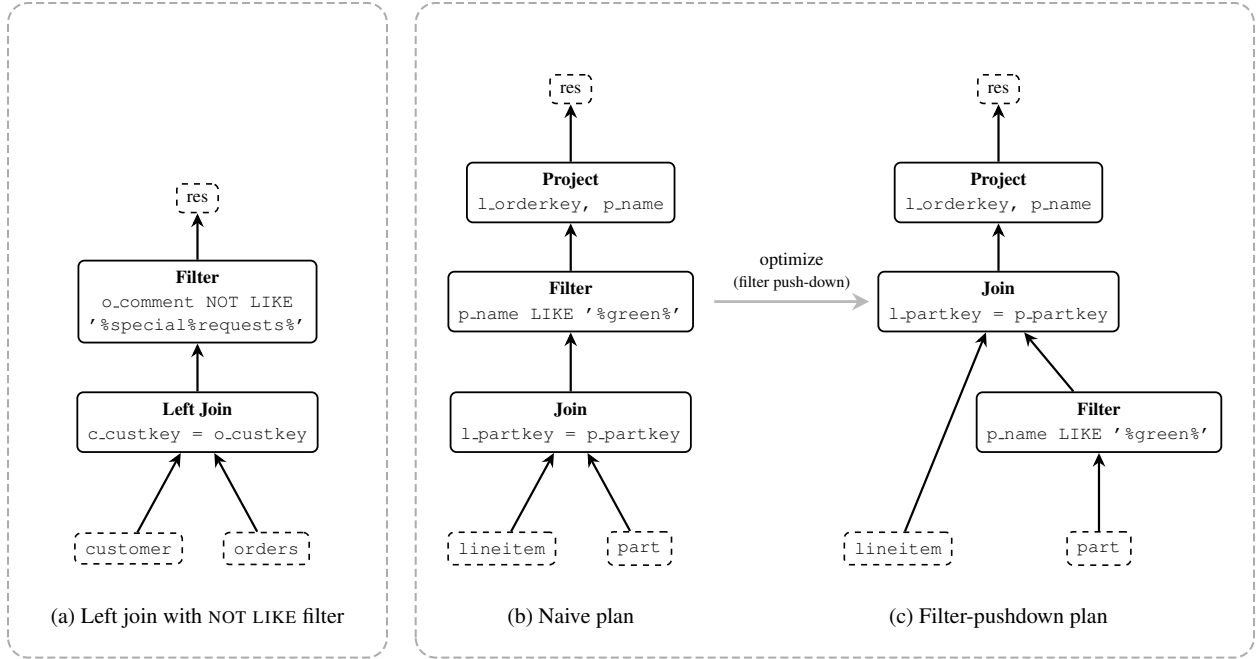


Figure 2: Two LIKE query plans: the filter in (a) cannot be pushed below the join, whereas the one in (b,c) can (b $\rightarrow$ c).

with the join; the optimizer therefore pushes it down to the base scan (b $\rightarrow$ c), letting the auxiliary columns be released before the join.

4 String Toolbox

We present a collection of protocols that serve as building blocks for the string-matching protocols of later sections, and that can be reused elsewhere as well.

4.1 Broadcast Check

In some protocols, we need to expand a string-level column into a character-level one (see Section 3.2 for definitions) by copying each string’s entry onto all of its characters; we call this operation *broadcasting*. We package this as the relation

$$\mathcal{R}_{\text{Bcast}} = \left\{ \begin{array}{l} \mathbb{x} = \perp, \\ \mathbb{y} = (\llbracket \text{orig-ind} \rrbracket, \llbracket \tilde{\text{d}} \rrbracket, \llbracket \tilde{\text{d}}' \rrbracket), \\ \mathbb{w} = (\text{orig-ind}, \text{d}, \text{d}') \end{array} \middle| \begin{array}{l} \text{d}'[c] = \text{d}[\text{orig-ind}[c]] \text{ for every character } c \end{array} \right\}.$$

The following PIOP checks this broadcast relationship.

PIOP 1: Broadcast Check

Inputs: $\llbracket \text{orig-ind} \rrbracket, \llbracket \text{ind} \rrbracket, \llbracket \tilde{\text{d}} \rrbracket, \llbracket \tilde{\text{d}}' \rrbracket$.

1. **V** samples a random $r \xleftarrow{\$} \mathbb{F}$ and sends it to **P**.
2. **P** and **V** invoke Lookup Check on $(1, \text{orig-ind} + r \text{d}') \sqsubseteq (1, \text{ind} + r \text{d})$.

4.2 Consistency-Checking protocols

The auxiliary columns must remain trusted and in sync with one another and with the main column at all times. Since the prover is untrusted, it cannot update them freely: every update must come with a proof of well-formedness. We therefore design protocols that prove this well-formedness.

Three kinds of updates arise in the system: *data-preserving* (Section 4.2.1), *domain-preserving* (Section 4.2.2) and *domain-altering*. The relational core of SQL falls almost entirely into the first two, with the third one confined to a handful of value-constructing expressions where all of the columns are freshly materialized and new strings are introduced: aggregates such as `STRING_AGG`, and string constructors such as `CONCAT`, `SUBSTRING`, case mapping, casts, and literals.

4.2.1 Data-Preserving Update

In a data-preserving update, only the activators are freshly materialized, i.e. columns `a` and `char-act`, while the remaining data columns are inherited unchanged. These are the operators that merely re-mark which rows are active—selection, projection, `DISTINCT`, and `LIMIT`.

Before relying on the new activators, the verifier must confirm that they are *consistent*. Recall that a string column carries a string-level activator `a`, marking which strings are active, and a character-level activator `char-act`, marking which characters are active. The pair is consistent when both columns are boolean and every character inherits the active bit of the string it belongs to. We package this as the relation

$$\mathcal{R}_{\text{Cons}} = \left\{ \begin{array}{l} \mathbb{x} = \perp, \\ \mathbb{y} = (\llbracket \text{char-act} \rrbracket, \llbracket \tilde{\text{a}} \rrbracket), \\ \mathbb{w} = (\text{char-act}, \text{a}) \end{array} \middle| \begin{array}{l} \text{char-act}[j], \text{a}[i] \in \{0, 1\} \text{ for all } i, j, \text{ and} \\ \text{char-act}[j] = \text{a}[i] \text{ for every character } j \text{ of string } i \end{array} \right\}.$$

which the following protocol establishes it.

PIOP 2: Data-Preserving Update Check

Inputs: $\llbracket \text{char-act} \rrbracket, \llbracket \tilde{a} \rrbracket, \llbracket \text{orig-ind} \rrbracket, \llbracket \text{ind} \rrbracket, \llbracket l \rrbracket$.

1. (*booleanity*) **P** and **V** invoke Booleanity Check on char-act and on a .
2. (*consistency*) **P** and **V** define the table L with key column $L.\text{key} = \text{orig-ind}$, value column $L.\text{val} = 1$, and activator $L.a = \text{char-act}$, and the table R with key column $R.\text{key} = \text{ind}$, value column $R.\text{val} = l$, and activator $R.a = a$. They then invoke KeyedSumcheck on L and R , which forces $\sum_{j: \text{orig-ind}[j]=i} \text{char-act}[j] = a[i] \cdot l[i]$ for every key i .

4.2.2 Domain-Preserving Update

In a domain-preserving update, all of the main and auxiliary columns, including data and activators, are freshly materialized by the untrusted prover, yet every output string cell is a copy of some input cell, so no new string is introduced. This covers all join types, UNION, INTERSECT, EXCEPT, sorting, and value-selecting aggregates such as MIN, MAX, and ANY_VALUE.

Since the prover materializes every new column, the check must establish two things: that the new columns form a well-formed arithmetization, and that no new string is introduced. The second requirement is the expensive one—verified directly, it would require re-hashing the freshly materialized characters—but we obtain it *for free* from the trusted old columns: matching each new string to its source row pins its hash, length, and characters to those of an old string, so the new hash is correct by inheritance and never recomputed. What does *not* come for free is the *structural* shape of the new columns—that l^{new} counts the characters of each new string and that $\text{int-ind}^{\text{new}}$ enumerates their within-word positions without gaps or repeats. These constraints are self-referential to the new columns, untethered to the old ones, and left unchecked they let the prover reorder, duplicate, or drop characters while still passing the source lookups. We therefore certify the structure directly with a handful of cheap checks—no hashing—and only then transfer the content by provenance.

PIOP 3: Full String Validity Check

Inputs: the old columns $\llbracket \tilde{h}^{\text{old}} \rrbracket, \llbracket l^{\text{old}} \rrbracket, \llbracket \tilde{a}^{\text{old}} \rrbracket, \llbracket \text{char}^{\text{old}} \rrbracket, \llbracket \text{orig-ind}^{\text{old}} \rrbracket, \llbracket \text{int-ind}^{\text{old}} \rrbracket, \llbracket \text{char-act}^{\text{old}} \rrbracket, \llbracket \text{bnd}^{\text{old}} \rrbracket$; the new columns $\llbracket \tilde{h}^{\text{new}} \rrbracket, \llbracket l^{\text{new}} \rrbracket, \llbracket \tilde{a}^{\text{new}} \rrbracket, \llbracket \text{char}^{\text{new}} \rrbracket, \llbracket \text{orig-ind}^{\text{new}} \rrbracket, \llbracket \text{int-ind}^{\text{new}} \rrbracket, \llbracket \text{char-act}^{\text{new}} \rrbracket, \llbracket \text{bnd}^{\text{new}} \rrbracket$; and potentially the source column $\llbracket \text{sfc} \rrbracket$.

1. (*Activators and lengths*) **P** and **V** invoke Data-Preserving Update Check on the new columns ($\text{char-act}^{\text{new}}, a^{\text{new}}, \text{orig-ind}^{\text{new}}, \text{ind}, l^{\text{new}}$), establishing $\mathcal{R}_{\text{Cons}}$ for the new activator pair.
2. (*Offset enumeration*) **V** samples $r \xleftarrow{\$} \mathbb{F}$ and sends it to **P**; the parties then invoke No-Duplicate Check on a column with data $\text{orig-ind}^{\text{new}} + r \cdot \text{int-ind}^{\text{new}}$ and activator $\text{char-act}^{\text{new}}$.
3. (*Source Column*) If there does not exist a trusted source column apriori:
 - (a) **P** creates $\llbracket \text{sfc} \rrbracket$ and sends it to **V**.
 - (b) **P** and **V** invoke Lookup Check on the statement $(\text{sfc}, \tilde{h}^{\text{new}}, l^{\text{new}}, \tilde{a}^{\text{new}}) \sqsubseteq (\text{id}_x, \tilde{h}^{\text{old}}, l^{\text{old}}, \tilde{a}^{\text{old}})$.
4. **P** broadcasts the sfc column of size N to the column sfc' of size M , then **P** and **V** invoke Broadcast Check on them.
5. **P** and **V** invoke Lookup Check on the statement $(\text{sfc}', \text{char}^{\text{new}}, \text{int-ind}^{\text{new}}) \sqsubseteq (\text{orig-ind}^{\text{old}}, \text{char}^{\text{old}}, \text{int-ind}^{\text{old}})$.

4.3 Length Filtering

Length filtering discards every string shorter than a threshold k , keeping only those of length at least k . This is the pruning needed in pattern matching: in every filter a string can match only if it is at least as long as the pattern, so any shorter string is a guaranteed non-match and can be removed before the filter runs. As

we will see in Section 5.1 and ??, this is not merely an optimization: length filtering is necessary for the pattern-matching protocols to remain complete and sound. We define the length filtering relation as follows 

$$\mathcal{R}_{\text{LenFilter}} = \left\{ \begin{array}{l} \mathbb{x} = k, \\ \mathbb{y} = (\llbracket \tilde{\mathbf{l}} \rrbracket, \llbracket \tilde{\mathbf{a}} \rrbracket, \llbracket \text{char-act} \rrbracket, \llbracket \tilde{\mathbf{a}}' \rrbracket, \llbracket \text{char-act}' \rrbracket), \\ \mathbb{w} = (l, a, \text{char-act}, a', \text{char-act}') \end{array} \middle| \begin{array}{l} a'[i] = 1 \iff (a[i] = 1 \text{ and } l[i] \geq k) \\ \text{char-act}'[j] = a'[i] \ \forall \text{ char } j \text{ of string } i \end{array} \right\}.$$

PIOP 4: Length Filtering Check

Inputs: $\llbracket \text{orig-ind} \rrbracket, \llbracket \tilde{\mathbf{l}} \rrbracket, \llbracket \tilde{\mathbf{a}} \rrbracket, \llbracket \text{char-act} \rrbracket$, threshold k .

1. (*Activator Update Validity*) $\mathbf{P}$ and $\mathbf{V}$ invoke Data-Preserving Update Check on $(\text{char-act}', a', \text{orig-ind}, \text{ind}, l)$.
2. (*Containment*) $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column $a' \cdot (1 - a)$.
3. (*False negative check*) $\mathbf{P}$ and $\mathbf{V}$ invoke the Negative Sign Check on the column $(a \cdot (1 - a'), l - k)$.
4. (*False Positive check*) $\mathbf{P}$ and $\mathbf{V}$ invoke the Non-Negative Sign Check on the column $(a', l - k)$.

5 PIOPs for Wild-Card Pattern Matching

In this section we develop PIOPs for SQL *wild-card* pattern matching — the predicates written with the `LIKE` operator. A `LIKE` pattern may use two kinds of wild-card: the multi-character `%`, which matches any (possibly empty) run of characters, and the single-character `_`, which matches exactly one character. Figure 3 shows the taxonomy of patterns we support: three kinds — single-character, multi-character, and mixed.

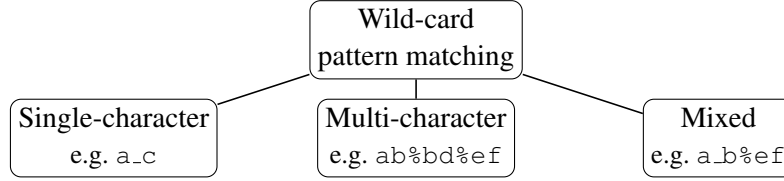


Figure 3: Taxonomy of wild-card pattern matching

5.1 Multi-Character

The most common kind of string pattern matching in natural SQL workloads is a multi-character pattern of the form $p_1\%p_2\%\dots\%p_t$, where each factor p_i is a literal string and `%` matches an arbitrary-length run of characters; the predicate holds iff the factors occur *in order* and *without overlap* within the value. For example, for the pattern “ab%bd”, the strings “abd” and “bdab” do not match: the former has an overlap, and the latter does not respect the ordering. The string “abcbd”, however, does satisfy the pattern.

In this section, we present Multi-Character Pattern Matching, a protocol that convinces the verifier that the activators were correctly updated from $\llbracket a^{\text{old}} \rrbracket$ (with its corresponding char-level activator $\llbracket \text{char-act}^{\text{old}} \rrbracket$) to $\llbracket a^{\text{new}} \rrbracket$ (with its corresponding char-level activator $\llbracket \text{char-act}^{\text{new}} \rrbracket$); that is, every string that is kept active satisfies the pattern (no false positives) and every string that is discarded does not (no false negatives).

We now describe the protocol at a high level.

Activator Update Validity. The prover and the verifier first check if the updated activators are consistent with each other. As mentioned in Section 4.2, the prover cannot update the string activators arbitrarily and needs to prove that all the main and auxiliary columns are consistent. This is done in Step 1.

Length Filtering. Before inspecting the patterns, we filter the strings by their length. This step drops every string whose length is smaller than k , since such a string cannot possibly satisfy the pattern. It not only reduces the number of strings—making the subsequent steps faster—but is also necessary for the completeness and soundness of the protocol. This is done in Step 2.

Containment. In any filtering operation, including this pattern matching, we must ensure that every element active in the output was already active in the input; that is, the filter does not introduce any new strings. This is done in Step 3.

Sweep & Verdict. We then sweep through all t factors, discarding at each round the strings that do not contain the current factor. Finally, the strings that remain alive are exactly those in which every factor was placed—that is, the strings that satisfy the entire pattern. The sweep records this survival information in the string-level activator alive_h , so alive_h marks precisely the matching strings. In the honest case it therefore equals a^{new} , which Step 5 verifies with a single zerocheck.

PIOP 5: Multi-Character Pattern Matching

Inputs: $\mathcal{P}$, $\llbracket a^{\text{old}} \rrbracket$, $\llbracket \text{char-act}^{\text{old}} \rrbracket$, $\llbracket a^{\text{new}} \rrbracket$, $\llbracket \text{char-act}^{\text{new}} \rrbracket$, $\llbracket \text{char} \rrbracket$, $\llbracket \text{bnd} \rrbracket$, $\llbracket \text{orig-ind} \rrbracket$, $\llbracket \text{int-ind} \rrbracket$, $\llbracket \text{ind} \rrbracket$, $\llbracket \tilde{I} \rrbracket$.

1. (*Activator validity*) **P** and **V** invoke Data-Preserving Update Check on $(\text{char-act}^{\text{new}}, \text{a}^{\text{new}}, \text{orig-ind}, \text{ind}, \text{l})$.
2. (*Length filtering*) **P** computes $\text{a}^{\text{old}'}$ and $\text{char-act}^{\text{old}'}$, the result of dropping strings shorter than k , and the parties invoke Length Filtering Check on $(\text{a}^{\text{old}}, \text{char-act}^{\text{old}}, \text{a}^{\text{old}'}, \text{char-act}^{\text{old}'})$ with threshold k .
3. (*Containment*) **P** and **V** invoke Zerocheck on $\text{a}^{\text{new}} \cdot (1 - \text{a}^{\text{old}'})$.
4. (*Sweep Factors*) **P** and **V** invoke Sweep Factors on $(\mathcal{P}, \text{a}^{\text{old}'}, \text{char-act}^{\text{old}'}, \llbracket \text{char} \rrbracket, \llbracket \text{bnd} \rrbracket, \llbracket \text{orig-ind} \rrbracket, \llbracket \text{int-ind} \rrbracket, \llbracket \text{ind} \rrbracket)$, which sweeps the factors $p_1, \dots, p_t$ in order and returns the final string-level activator alive_h .
5. (*Verdict*) **P** and **V** invoke Zerocheck on $\text{a}^{\text{new}} - \text{alive}_h$.

Next we discuss the main building block of Multi-Character Pattern Matching, namely Sweep Factors, which carries out the entire sweep and returns the final string-level activator alive_h , marking exactly the strings that contain all t factors in order and without overlap.

Easy case: Single Factor. We first explain how to check if a column of strings satisfy a single factor $p = (\text{str}, \text{mode})$, then extend it to a multi-factor scenario. Let $\ell = |\text{str}|$, then the prover needs to convince the verifier that (i) there exist a *window of length ℓ* that has the exact pattern str , and (ii) there does not exist such a window for the discarded strings. Also the start position of this window matters. For $\text{mode} = \text{prefix}$, we want this window to start at the 0th location of the survived strings, for $\text{mode} = \text{suffix}$, we want this window to start at the last location of the survived strings, and for $\text{mode} = \text{infix}$, the window can start from anywhere inside the string.

To reason about a “window” of characters and its location, we introduce a *window fingerprint* column. The prover forms the ℓ shifted copies $\text{char}^{(0)}, \dots, \text{char}^{(\ell-1)}$ of the character column and combines them into a single column $\text{wf} := \sum_{\delta=0}^{\ell-1} r_\delta \text{char}^{(\delta)}$ using random coefficients r_δ supplied by the verifier, so that $\text{wf}[c]$ is a fingerprint of the length- ℓ window starting at position c . The prover and verifier then fingerprint the pattern itself in the same way, computing the *pattern fingerprint* $\text{pf} := \sum_{\delta=0}^{\ell-1} r_\delta \text{str}[\delta]$. With both fingerprints in hand, matching a single factor reduces to a numerical equality test: wherever $\text{wf}[c] = \text{pf}$, the pattern occurs starting at position c .

As noted, the valid starting location of the match depends on the factor’s mode, so we introduce an additional witness called the *attention mask* that tells the verifier where a match is allowed to begin. For a prefix, the attention mask is 1 at the beginning of each string (i.e., the boundary-marker column); for a suffix, it is 1 at the end of each string (i.e., a rotation of the boundary-marker column); and for an infix, it may be 1 at any interior position, subject to additional constraints. These masks are set up in Step 2.

From a Single-Factor to Multi-Factor. To check whether the factors $\mathcal{P} = \{p_1, \dots, p_t\}$ occur in order and without overlap, we first prepare the rotated character polynomials for future fingerprintings in Step 1. Next, we explain how the actual sweeping happens in a high level.

Sweeping Overview. We sweep the factors $p_1, \dots, p_t$ from left to right in t rounds: one factor per round. Round j locates the leftmost occurrence of the factor p_j within the each string using Factor Placement and forms two string-level columns: (i) match_j , a boolean indicator of whether the string contains p_j at all; and (ii) start_j , the internal index at which that leftmost occurrence begins. This is done in Step 4.

The key idea is that once p_j has been placed, every character up to and including that occurrence is switched off, so the next factor is searched only within the remaining suffix. A single character-level activator alive_c tracks this shrinking suffix, and it is what enforces the ordering and non-overlap requirements simultaneously: a later factor can only be placed strictly after an earlier one. Committing to the *leftmost* occurrence is safe because it leaves the largest possible suffix for the factors that follow.

For example, consider the pattern “%ab%bd%t” and the string “kabyabrbdut” with characters $[k, a, b, y, a, b, r, b, d, u, t]$. Initially every character is active, so $\text{alive}_c^{(0)} = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]$. The first round places “ab” at its leftmost occurrence and switches off everything up to and including it, leaving $\text{alive}_c^{(1)} = [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1]$; only the characters after “ab” remain under consideration. The second

round places “bd” within that suffix, giving $\text{alive}_c^{(2)} = [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1]$, so the final factor “t” is searched only in the last two positions—where it is indeed found, and the string matches.

PIOP 6: Sweep Factors

Inputs: the ordered factors $\mathcal{P} = (p_1, \dots, p_t)$ with $p_j = (\text{str}_j, \text{mode}_j)$; the initial activators $\llbracket \tilde{a}^{\text{old}'} \rrbracket$ and $\llbracket \text{char-act}^{\text{old}'} \rrbracket$; and the fixed columns $\llbracket \tilde{\text{char}} \rrbracket, \llbracket \tilde{\text{bnd}} \rrbracket, \llbracket \tilde{\text{orig-ind}} \rrbracket, \llbracket \tilde{\text{int-ind}} \rrbracket, \llbracket \tilde{\text{ind}} \rrbracket$.

1. (*Initialization*) $\mathbf{P}$ and $\mathbf{V}$ set $\text{alive}_h^{(0)} := a^{\text{old}'}$ and $\text{alive}_c^{(0)} := \text{char-act}^{\text{old}'}$. They then prepare the window data shared by every round: writing $\ell_{\max} := \max_j |\text{str}_j|$ for the maximum factor length, $\mathbf{P}$ computes the rotated character columns $\text{char}^{(1)}, \dots, \text{char}^{(\ell_{\max}-1)}$ and sends them to $\mathbf{V}$, where $\text{char}^{(0)} := \text{char}$ and each $\text{char}^{(\delta)}$ is a rotation of $\text{char}^{(\delta-1)}$ by -1 (so $\text{char}^{(\delta)}[p] = \text{char}[p + \delta]$); for $\delta = 1, \dots, \ell_{\max} - 1$ the parties invoke Rotation Check on $(\text{char}^{(\delta-1)}, \text{char}^{(\delta)}, -1)$. Finally $\mathbf{V}$ samples $r_0, \dots, r_{\ell_{\max}-1} \xleftarrow{\$} \mathbb{F}$.
2. (*Sweep*) For $j = 1, \dots, t$ in order, writing $\ell := |\text{str}_j|$, $\mathbf{P}$ and $\mathbf{V}$ carry out the following on the region delimited by $\text{alive}_h^{(j-1)}$ and $\text{alive}_c^{(j-1)}$:
 - (a) (*Factor placement*) the parties invoke Factor Placement on $(p_j, \text{alive}_h^{(j-1)}, \text{alive}_c^{(j-1)}, \{\llbracket \tilde{\text{char}}^{(\delta)} \rrbracket\}_{\delta=0}^{\ell-1}, (r_\delta)_{\delta=0}^{\ell-1}, \llbracket \tilde{\text{bnd}} \rrbracket, \llbracket \tilde{\text{orig-ind}} \rrbracket, \llbracket \tilde{\text{int-ind}} \rrbracket, \llbracket \tilde{\text{ind}} \rrbracket)$, obtaining the match indicator match_j and placement start_j ;
 - (b) (*Past-placement indicator*) $\mathbf{P}$ broadcasts start_j to character level as start'_j via Broadcast Check, and the parties invoke the Non-Negative Sign Check on $\text{int-ind}[c] - \text{start}'_j[c] - |p_j|$, receiving the indicator past_j with $\text{past}_j[c] = \mathbf{1}[\text{int-ind}[c] \geq \text{start}'_j[c] + |p_j|]$.
 - (c) (*Updating the activators*) $\mathbf{P}$ and $\mathbf{V}$ set $\text{alive}_h^{(j)}[i] := \text{alive}_h^{(j-1)}[i] \cdot \text{match}_j[i]$, and $\mathbf{P}$ broadcasts it to character level as $\text{alive}_h^{(j)'}$ via Broadcast Check. $\mathbf{P}$ then forms $\text{alive}_c^{(j)}[c] := \text{alive}_c^{(j-1)}[c] \cdot \text{alive}_h^{(j)'}[c] \cdot \text{past}_j[c]$, and the parties invoke a Zerocheck on $\text{alive}_c^{(j)} - \text{alive}_c^{(j-1)} \cdot \text{alive}_h^{(j)'} \cdot \text{past}_j$.
3. (*Output*) After the t rounds, $\mathbf{P}$ and $\mathbf{V}$ output the final string-level activator $\text{alive}_h^{(t)}$.

The per-round placement invoked above is carried out by Factor Placement, which uses the same fingerprinting and attention-mask machinery as the single-factor case, the difference being that it must additionally pin down the *leftmost* occurrence of the factor. To this end, in Step 3 the prover first sends an occurrence column recording *all* occurrences of the factor: the verifier must know every occurrence before it can check that the prover selected the leftmost one among them.

Leftmost Occurrence. To confirm that the start column output by Factor Placement indeed marks the leftmost occurrence, the verifier checks, in Step 4, that no occurrence precedes the one recorded in start.

PIOP 7: Factor Placement

Inputs: $p = (\text{str}, \text{mode})$ with $\ell = |\text{str}|$; the input activators $\llbracket \tilde{a} \rrbracket$ and $\llbracket \tilde{\text{char-act}} \rrbracket$; the rotated character columns $\{\llbracket \tilde{\text{char}}^{(\delta)} \rrbracket\}_{\delta=0}^{\ell-1}$ and coefficients $r_0, \dots, r_{\ell-1}$; and $\llbracket \tilde{\text{bnd}} \rrbracket, \llbracket \tilde{\text{orig-ind}} \rrbracket, \llbracket \tilde{\text{int-ind}} \rrbracket, \llbracket \tilde{\text{ind}} \rrbracket$.

Output: the string-level match indicator match and placement start.

1. (*Window fingerprinting*) using the rotated columns and coefficients supplied as input, the parties form $\text{wf} := \sum_{\delta=0}^{\ell-1} r_\delta \text{char}^{(\delta)}$, $\text{pf} := \sum_{\delta=0}^{\ell-1} r_\delta \text{str}[\delta]$, and $\text{diff} := \text{wf} - \text{pf}$; this is a virtual linear combination, so no new commitments or rotation checks are needed.
2. (*Attention mask*) the parties fix att_mask according to mode:
 - (a) *prefix:* $\text{att_mask} := \text{char-act} \cdot \text{bnd}$;
 - (b) *suffix:* $\mathbf{P}$ computes $\text{att_mask} := \rho_{-\ell}(\text{char-act} \cdot \text{bnd})$ and sends it to $\mathbf{V}$, and the parties invoke Rotation Check on $(\text{char-act} \cdot \text{bnd}, \text{att_mask}, -\ell)$;
 - (c) *infix:* $\mathbf{P}$ computes the rotated boundary markers $\text{bnd}^{(1)}, \dots, \text{bnd}^{(\ell-1)}$ and sends them to $\mathbf{V}$, where $\text{bnd}^{(0)} := \text{bnd}$ and each $\text{bnd}^{(\delta)}$ is a rotation of $\text{bnd}^{(\delta-1)}$ by -1 (so $\text{bnd}^{(\delta)}[i] = \text{bnd}[i + \delta]$); the parties invoke Rotation Check for each $(\text{bnd}^{(\delta-1)}, \text{bnd}^{(\delta)}, -1)$ and set $\text{att_mask} := \text{char-act} \cdot (1 - \sum_{\delta=1}^{\ell-1} \text{bnd}^{(\delta)})$.

3. (*Occurrence column*) $\mathbf{P}$ computes and sends `occurs`, which is 1 at the start of every occurrence of `str` among the candidate windows, and the parties establish its validity by
 - (a) a Booleanity Check on `occurs`;
 - (b) a Zerocheck on `occurs · (1 - att_mask)`, so every mark is a candidate window;
 - (c) a Zerocheck on `occurs · diff`, so every marked window equals `str`;
 - (d) a NoZero Check on the column with data `diff` and activator `att_mask · (1 - occurs)`, so every unmarked candidate differs from `str`,
 forcing `occurs` to mark exactly the candidate windows equal to `str`.
4. (*Leftmost placement*) Writing $O_i := \{ \text{int-ind}[c] : \text{orig-ind}[c] = i, \text{occurs}[c] = 1 \}$ for the occurrence positions in string i , $\mathbf{P}$ builds and sends
 - the string-level indicator $\text{match}[i] := \mathbf{1}[O_i \neq \emptyset]$ (the string carries an occurrence);
 - the placement $\text{start}[i] := \min O_i$ when $\text{match}[i] = 1$ (the leftmost occurrence), and $\text{start}[i] := 0$ otherwise;
 - the character-level marker $\text{mark}[c] := \mathbf{1}[\text{occurs}[c] = 1 \text{ and } \text{int-ind}[c] = \text{start}[\text{orig-ind}[c]]]$ (the character at that leftmost occurrence).
 The parties then check
 - (a) a Booleanity Check on `match` and on `mark`;
 - (b) (*no false negatives*) $\mathbf{P}$ broadcasts `match` to character level (`match'`) via Broadcast Check, and the parties invoke a Zerocheck on `occurs · (1 - match')`, so every occurrence lies in a matched string;
 - (c) (*no false positives*) a Zerocheck on `mark · (1 - occurs)` (each mark is an occurrence), a No-Duplicate Check on `orig-ind` with activator `mark` (at most one mark per string), and—with a count n_m from $\mathbf{P}$ —two Sumchecks for $\sum_c \text{mark}[c] = n_m$ and $\sum_i \text{match}[i] = n_m$, so exactly one occurrence is marked in each matched string;
 - (d) (*placement*) $\mathbf{V}$ samples $\gamma \xleftarrow{\$} \mathbb{F}$ and the parties invoke Lookup Check on $(\text{match}, \text{ind} + \gamma \text{start}) \sqsubseteq (\text{mark}, \text{orig-ind} + \gamma \text{int-ind})$, i.e. with `match` activating the string-level column `ind + γ start` and `mark` activating the character-level column `orig-ind + γ int-ind`. This ties each matched string's start to the `int-ind` of its mark (both activated sides have n_m elements, so the lookup forces equality);
 - (e) (*leftmost*) $\mathbf{P}$ broadcasts `start` to character level (`start'`); a Sign Check forms $\mathbf{1}[\text{int-ind} < \text{start}']$, and the parties invoke a Zerocheck on `occurs · 1[int-ind < start']` with activator `match'`, so no occurrence precedes the marked one.

6 Pre-Filtering Optimization

In Section 5 we presented a PIOP for pattern matching that is proven secure and ready to be deployed. Although Multi-Character Pattern Matching proves what is conceptually a simple filter, it operates on the full character-level machinery of the string arithmetization and is therefore expensive. This motivates us to shrink the string columns fed into it as much as possible.

To this end, we borrow the *fingerprinting* technique from the database literature, where it was introduced [STZRK25] as a fast pre-filter for query execution: it cheaply discards *true negatives*, the rows that certainly do not match the pattern, while possibly retaining *false positives*, rows that survive the filter but do not actually match. Once most true negatives are gone, the surviving column is small enough that an exact but expensive pattern-matching protocol can affordably remove the remaining false positives.

Although fingerprinting was originally proposed to speed up the *execution* of pattern matching, we find that it speeds up its *verification* just as effectively. Before presenting the verification mechanism, we first describe how fingerprint-based pre-filtering works.

6.1 Pre-Filtering via Fingerprinting

The idea of fingerprinting is to replace the full representation of a string with a handful of *bins*, each bin holding a single bit that is 1 if and only if the string satisfies a fixed property. The fingerprint of a string is the vector of its bin values.

For example, suppose we have $m = 4$ bins, where bin $i \in \{1, 2, 3, 4\}$ is computed by its own predicate $\text{fp}_i : \text{Strings} \rightarrow \{0, 1\}$ defined as follows:

$$\begin{aligned} \text{fp}_1(s) &= \begin{cases} 1 & \text{if } s \text{ contains 'a', 'b', or 'c'} \\ 0 & \text{otherwise} \end{cases} & \text{fp}_2(s) &= \begin{cases} 1 & \text{if } s \text{ contains any of 'd', 'e', ..., 'l'} \\ 0 & \text{otherwise} \end{cases} \\ \text{fp}_3(s) &= \begin{cases} 1 & \text{if } s \text{ contains 'm' or 'n'} \\ 0 & \text{otherwise} \end{cases} & \text{fp}_4(s) &= \begin{cases} 1 & \text{if } s \text{ contains any of 'o', 'p', ..., 'z'} \\ 0 & \text{otherwise} \end{cases} \end{aligned}$$

Now suppose we want to evaluate the filter `%dar%` on the strings `darling`, `zebra`, and `lemon`. Fingerprinting the pattern's literal `dar` and the three strings yields

$$\text{dar} \rightarrow 1101, \quad \text{darling} \rightarrow 1111, \quad \text{zebra} \rightarrow 1101, \quad \text{lemon} \rightarrow 0111.$$

A string can match `%dar%` only if it contains every character of `dar`, so every bin that is active in the pattern's fingerprint must also be active in the string's fingerprint. The first bin is active in the pattern but inactive in `lemon`: `lemon` contains none of 'a', 'b', 'c', so it cannot contain `dar` and is discarded immediately. Both `darling` and `zebra`, on the other hand, activate every bin that the pattern does, so both survive the filter. Of these, `darling` is a genuine match, whereas `zebra` does not satisfy the predicate and is a *false positive* that only the exact protocol can remove.

6.2 A PIOP for Pre-Filtering

Fingerprints as field elements. We pack the m bins of a fingerprint into a single field element by reading the bin vector as an m -bit unsigned integer, bin 1 being the most significant bit:

$$\text{fp}(s) := \sum_{i=1}^m 2^{m-i} \text{fp}_i(s) \in [0, 2^m - 1].$$

A pattern ζ is fingerprinted through its literal characters only: the wild-cards are dropped and the characters of all factors are pooled, so $\text{fp}(\zeta)$ is the fingerprint of the string obtained by concatenating the factors. Throughout this subsection we write $\phi := \text{fp}(\zeta)$ for this public constant. Since every character of ζ must appear in any string that matches it, the subset condition of the previous subsection is precisely the bitwise identity

$$\text{fp}(s) \wedge \phi = \phi,$$

where $\wedge$ denotes bitwise AND. This is the operation that the Bitwise AND check of Section 6.3 proves, and it is the sole non-linear ingredient of the protocol below.

The fingerprint column. We extend the string arithmetization of Section 3.2 with one more string-level auxiliary column, the *fingerprint column* fp , with $\text{fp}[i] = \text{fp}(c[i])$ the m -bit fingerprint of the i -th string. Like the length column l , it is computed once by data owner and then carried along by the consistency-checking protocols of Section 4.2: a data-preserving update inherits it unchanged, and a domain-preserving update copies it from the source row through the same lookup that transfers h and l . Hence, by the time a pattern-matching query arrives, fp is trusted.

The relation. Pre-filtering is itself a data-preserving update: the prover freshly materializes the activators a' and $\text{char-act}'$, keeping exactly the active strings whose fingerprint passes the subset test, and every other column is inherited. We package this as the relation

$$\mathcal{R}_{\text{PreFilter}} = \left\{ \begin{array}{l} \mathbf{x} = (m, \phi), \\ \mathbf{y} = ([\tilde{\text{fp}}], [\tilde{a}], [\tilde{\text{char-act}}], [\tilde{a}'], [\tilde{\text{char-act}}']), \\ \mathbf{w} = (\text{fp}, a, \text{char-act}, a', \text{char-act}') \end{array} \middle| \begin{array}{l} a'[i] = 1 \iff (a[i] = 1 \text{ and } \text{fp}[i] \wedge \phi = \phi) \\ \text{char-act}'[j] = a'[i] \ \forall \text{ char } j \text{ of string } i \end{array} \right\}.$$

The protocol. The first two steps are the standard ones for any filter: the new activator pair must be consistent (Data-Preserving Update Check), and no inactive string may be revived (a zerocheck on $a' \cdot (1 - a)$). The remaining steps enforce the fingerprint test. The prover sends the *masked column* $g := \text{fp} \wedge \phi$, obtained by AND-ing every fingerprint with the pattern fingerprint, and the parties certify it with the Bitwise AND check applied to fp , the constant column $\phi \cdot 1$, and g ; the constant column needs no oracle, as its multilinear extension is the constant ϕ , which the verifier evaluates by itself. Once g is certified, the test $\text{fp}[i] \wedge \phi = \phi$ becomes the equality $g[i] = \phi$, so $g - \phi$ is zero exactly on the strings that pass and nonzero on every string that fails. The verdict therefore splits into two complementary checks, as in length filtering: a zerocheck on $a' \cdot (g - \phi)$ rules out false positives (no kept string fails the test), and a NoZero check on $g - \phi$ activated by $a \cdot (1 - a')$ rules out false negatives (every dropped string genuinely fails). Unlike length filtering, no order comparison is needed here, so the cheaper NoZero check replaces the sign check.

PIOP 8: Pre-Filtering Check

Inputs: $[\tilde{\text{fp}}], [\tilde{a}], [\tilde{\text{char-act}}], [\tilde{a}'], [\tilde{\text{char-act}}'], [\text{orig-ind}], [\text{ind}], [\tilde{l}]$, bin count m , pattern fingerprint ϕ .

1. (*Activator Update Validity*) $\mathbf{P}$ and $\mathbf{V}$ invoke Data-Preserving Update Check on $(\text{char-act}', a', \text{orig-ind}, \text{ind}, l)$.
2. (*Containment*) $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column $a' \cdot (1 - a)$.
3. (*Masking*) $\mathbf{P}$ computes the masked column $g := \text{fp} \wedge \phi$ and sends $[\tilde{g}]$ to $\mathbf{V}$. The parties invoke BitwiseAND on $(\text{fp}, \phi \cdot 1, g)$ with bit-width $n = m$.
4. (*False Positive check*) $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column $a' \cdot (g - \phi)$.
5. (*False Negative check*) $\mathbf{P}$ and $\mathbf{V}$ invoke NoZero Check on the column with data $g - \phi$ and activator $a \cdot (1 - a')$.

6.3 Bitwise AND

The *Bitwise AND check* takes three columns c_1, c_2, c_3 containing n -bit values and proves that every entry of c_3 is the bitwise AND of the corresponding entries in c_1 and c_2 . We package this as the relation

$$\mathcal{R}_{\text{BitwiseAND}} = \{ \mathbb{x} = n, \mathbb{y} = ([\tilde{c}_1], [\tilde{c}_2], [\tilde{c}_3]), \mathbb{w} = (c_1, c_2, c_3) \mid \forall i, c_3[i] = c_1[i] \wedge c_2[i] \}.$$

$\mathcal{I}'_2(X_0, X_1, X_2, X_3) = 2X_0 + X_1$		$\mathcal{I}''_2(X_0, X_1, X_2, X_3) = 2X_2 + X_3$		$\tilde{\text{and}}(X_0, X_1, X_2, X_3) = 2X_0X_2 + X_1X_3$
0 (00)		0 (00)		0 (00)
0 (00)		1 (01)		0 (00)
0 (00)		2 (10)		0 (00)
0 (00)		3 (11)		0 (00)
1 (01)		0 (00)		0 (00)
1 (01)		1 (01)		1 (01)
1 (01)		2 (10)		0 (00)
1 (01)		3 (11)		1 (01)
2 (10)		0 (00)		0 (00)
2 (10)		1 (01)		0 (00)
2 (10)		2 (10)		2 (10)
2 (10)		3 (11)		2 (10)
3 (11)		0 (00)		0 (00)
3 (11)		1 (01)		1 (01)
3 (11)		2 (10)		2 (10)
3 (11)		3 (11)		3 (11)

Figure 4: The arithmetized truth-table for the bitwise AND function on $n=2$ bit inputs.

To convince the verifier that each row of c_3 is the bitwise AND of the corresponding rows of c_1 and c_2 , the prover will convince the verifier that the concatenated rows of c_1, c_2, c_3 can be looked up in a truth table encoding the bitwise AND function. More concretely,

PIOP 9: BitwiseAND

1. **P** and **V** invoke Range Check on each of the three columns to check that they are all n -bit.
2. **V** samples three random field elements r_1, r_2, r_3 and sends them to **P**.
3. **P** and **V** invoke Lookup Check to check that the concatenated column $c' = r_1c_1 + r_2c_2 + r_3c_3$ can be looked up in the column $c'' = r_1\mathcal{I}'_n + r_2\mathcal{I}''_n + r_3\tilde{o}$ where

$$\begin{aligned} \mathcal{I}'_n(X_0, \dots, X_{n-1}) &= \sum_{i=0}^{n-1} 2^{n-1-i} X_i, \\ \mathcal{I}''_n(X_0, \dots, X_{n-1}) &= \sum_{i=0}^{n-1} 2^{n-1-i} X_{n+i}, \\ \tilde{\text{and}}(X_0, \dots, X_{2n-1}) &= \sum_{i=0}^{n-1} 2^{n-1-i} X_i X_{n+i}. \end{aligned}$$

7 Implementation and Evaluation

7.1 Implementation

We implement TruthTable++ by extending the open-source implementation of TruthTable [NSSMM26] rather than building a new system. TruthTable is realized as two Rust libraries: `truth-table`¹, an end-to-end verifiable query execution system of about 60K lines of Rust, and `ark-piop`², the underlying PIOP infrastructure of about 14K lines on which it relies. TruthTable arithmetizes every column as an activator and a data vector, compiles each relational operator to a PIOP, and reduces all PIOPs to a common set of fundamental claims (sumcheck, zerocheck, nonzerocheck, evaluation, and lookup) that `ark-piop` batches and discharges. Its query planner extends the industry-grade planner of Apache DataFusion [Lam+24], and therefore inherits DataFusion’s suite of query optimizations.

TruthTable++ expands this code base to support string-typed columns, adding about 8K lines of Rust to `truth-table` and about 3K lines to `ark-piop`. Concretely, we add the following on top of TruthTable:

- *String arithmetization.* A string-typed column is materialized as the hash column of TruthTable augmented with the string-level and character-level auxiliary columns of Section 3.2, together with the fingerprint column of Section 6.2. All auxiliary columns are computed once at ingestion and committed alongside the main column.
- *String protocols.* The string toolbox of Sections 4.2 and 4.3, which every string-aware operator invokes to keep the auxiliary columns in sync with the main column; the wild-card pattern-matching PIOPs of Section 5, exposed to the planner as a `LIKE` predicate; and the pre-filtering and bitwise AND checks of Sections 6.2 and 6.3, which the planner inserts in front of every `LIKE` predicate. The bin partition of the alphabet is a per-column configuration parameter chosen at ingestion, e.g. from the instance-optimized partitions of [STZRK25].

7.2 Evaluation

Experimental setup. As in TruthTable, we instantiate TruthTable++ with the PST polynomial commitment scheme [PST13] over the BN254 elliptic curve. We run all experiments on a machine with a 13th Gen Intel Core i7-13700 CPU with 16 hardware threads and 64 GB of RAM, and report single-threaded prover time, verifier time, and proof size unless stated otherwise.

[Ali: Benchmarks and results to be added: (i) the TPC-H queries with `LIKE` predicates that TruthTable omitted (Q9, Q13, Q14, Q16, Q20), (ii) microbenchmarks on synthetic string columns varying the column size, the pattern length and number of factors, and (iii) the effect of pre-filtering on the prover time of Multi-Character Pattern Matching as a function of the bin count m and the false-positive rate.]

¹<https://github.com/arkworks-rs/truth-table>

²<https://github.com/alireza-shirzad/ark-piop>

Acknowledgments

A Extended Preliminaries

A.1 s-Contiguous Polynomial Identity

Here we show that the s-contiguous polynomial, defined in Section 2 as:

$$\mathcal{C}_{\mu,s}(X) := \sum_{i=1}^{\mu} s_i(1 - X_i) \prod_{j=1}^{i-1} (s_j X_j + (1 - s_j)(1 - X_j))$$

is 1 for the first s points in $\mathcal{H}_{\mu}$ and 0 elsewhere. Here, $s_i = \lfloor \frac{s}{2^{\mu-i}} \rfloor \bmod 2$ is the i -th most significant bit of s . Equivalently, $\mathcal{C}_{\mu,s}(X) = 1$ if and only if $\mathcal{I}_{\mu}(X) < s$, and $\mathcal{C}_{\mu,s}(X) = 0$ otherwise. Here, we will provide a short derivation of this identity.

Observe that the condition $\mathcal{I}_{\mu}(X) < s$ is satisfied if and only if there is some position i such that:

1. **Prefix Match:** For all indices $j < i$ (bits more significant than i), the bit X_j is identical to s_j .
2. **Current Bit Lower:** At index i , the bit X_i is strictly less than s_i . Since $X_i, s_i \in \{0, 1\}$, this forces $s_i = 1$ and $X_i = 0$.

The polynomial captures this logic by summing over all possible positions i where this divergence could occur. The inner product, $\prod_{j=1}^{i-1} (s_j X_j + (1 - s_j)(1 - X_j))$, acts as an equality indicator for the prefix, evaluating to 1 if and only if $X_j = s_j$ for all $j < i$. The term $s_i(1 - X_i)$ enforces the strict inequality at the current bit. Because the “first position of difference” is unique for any distinct pair of integers, these conditions are mutually exclusive across different i . Thus, the summation yields exactly 1 if such an i exists and 0 otherwise.

A.2 Deferred Low-Level PIOPs

Sumcheck. The idea of sumcheck is to reduce an exponential sum to a sequence of univariate sums, each involving one fewer variable. At each step, the prover sends a univariate polynomial that partially sums over the remaining variables. The verifier checks consistency between rounds by evaluating each polynomial at a random point and verifying that the sum matches the previous claim. After reducing to a single point, the verifier queries the original oracle to confirm the final value.

PIOP 10: Sumcheck

Inputs: $\llbracket p \rrbracket, s$.

1. **P** sends to **V** a univariate polynomial oracle p_1 such that $p_1(X_1) = \sum_{\{x_2, \dots, x_{\mu}\} \in \mathcal{H}_{\mu-1}} p(X_1, x_2, \dots, x_{\mu})$.
2. **V** checks that the degree of $p_1(x_1)$ is at most the degree of p in x_1 . It computes $s_1 = p_1(0) + p_1(1)$ and checks that $s_1 = s$. If not, **V** rejects. It then picks a random $r_1 \in \mathbb{F}$ and sends it to **P**.
3. For each i from 2 to μ (subsequent rounds):
 - (a) **P** sends to **V** the univariate polynomial $p_i(x_i) = \sum_{\{x_{i+1}, \dots, x_{\mu}\} \in \mathcal{H}_{\mu-i}} p(r_1, \dots, r_{i-1}, x_i, \dots, x_{\mu})$.
 - (b) **V** checks that the degree of $p_i(x_i)$ is at most the degree of p in x_i . It computes $s_i = p_i(0) + p_i(1)$ and checks that $s_i = p_{i-1}(r_{i-1})$. If not, **V** rejects. It picks a random $r_i \in \mathbb{F}$ and sends it to **P**.
4. **P** sends the value $p(r_1, \dots, r_{\mu})$ for the random values $r_1, \dots, r_{\mu}$ chosen by **V**.
5. **V** checks that this value is correct by querying p at $(r_1, \dots, r_{\mu})$.

Zerocheck. By the Schwartz–Zippel lemma, it suffices for the prover to convince the verifier that p is zero on a random point. This check is executed via a sumcheck over the polynomial $p(x)\text{Eq}(x, r)$. Throughout this paper we use this style of PIOP composition, enabling us to reason about more complex polynomial properties while growing our toolbox.

PIOP 11: Zerocheck

Inputs: $\llbracket p \rrbracket$.

1. **V** samples a vector $r \xleftarrow{\$} \mathbb{F}^\mu$ and sends it to **P**.
2. **P** and **V** invoke Sumcheck for the claim that $\sum_{x \in \mathcal{H}_\mu} p(x) \cdot \text{Eq}(x, r) = 0$.

Rational Sumcheck. The *rational sumcheck* takes two columns p, q and a claimed sum s , and proves that $\sum_{x \in \mathcal{H}_\mu} p(x)/q(x) = s$ under the promise that $q(x) \neq 0$ for all $x \in \mathcal{H}_\mu$. We package this as the relation

$$\mathcal{R}_{\text{RatSum}} = \left\{ \mathbb{x} = (\mu, s), \mathbb{y} = (\llbracket \tilde{p} \rrbracket, \llbracket \tilde{q} \rrbracket), \mathbb{w} = (p, q) \mid \sum_{x \in \mathcal{H}_\mu} \frac{p(x)}{q(x)} = s \right\}.$$

The prover constructs a polynomial $h(x)$ that equals $p(x)/q(x)$ on $\mathcal{H}_\mu$ and sends its oracle to the verifier, who can then be convinced that h behaves like the intended rational function via a Zerocheck. With a verified h in hand, the parties reduce the problem to a standard Sumcheck over h .

PIOP 12: RatSumcheck

Inputs: $\llbracket p \rrbracket, \llbracket q \rrbracket, s$.

1. **P** constructs the polynomial h such that for all $x \in \mathcal{H}_\mu$, $h(x) = \frac{p(x)}{q(x)}$, and sends its oracle $\llbracket h \rrbracket$ to **V**.
2. **P** and **V** invoke Zerocheck for the claim that $\forall x \in \mathcal{H}_\mu, h(x)q(x) - p(x) = 0$.
3. **P** and **V** invoke Sumcheck for the claim $\sum_{x \in \mathcal{H}_\mu} h(x) = s$.

B Deferred PIOPs

This appendix collects the low-level PIOPs and relations used throughout TruthTable++. These are standard or lightly adapted from prior work, and are deferred here so that the main text can focus on the string-filtering protocols.

B.1 NoZero Check

The *NoZero Check* takes a column $c = (d, a)$ and proves that every active entry of c is nonzero. We package this as the relation

$$\mathcal{R}_{\text{NoZero}} = \{ \mathbb{x} = n, \mathbb{y} = (\llbracket \tilde{c} \rrbracket), \mathbb{w} = (c) \mid \forall i, a[i] = 1 \implies c[i] \neq 0 \}.$$

A field element is nonzero exactly when it has a multiplicative inverse. The prover sends an inverse column inv whose value at each active index is the inverse of the data there and which is 0 at inactive indices. If inv is formed this way, then $d \cdot \text{inv} = a$ holds across the whole hypercube: at an active index both sides equal 1, and at an inactive index both sides equal 0 (since inv is 0 there). A single Zerocheck on $d \cdot \text{inv} - a$ certifies this, and forces every active entry to be invertible, hence nonzero.

PIOP 13: NoZero Check

Inputs: $\llbracket \tilde{c} \rrbracket = (\llbracket \tilde{a} \rrbracket, \llbracket \tilde{d} \rrbracket)$.

1. **P** constructs and sends the inverse oracle $\llbracket \tilde{\text{inv}} \rrbracket$, with $\text{inv}[i] = d[i]^{-1}$ at active indices and $\text{inv}[i] = 0$ elsewhere.
2. **P** and **V** invoke Zerocheck on the column $d \cdot \text{inv} - a$.

B.2 Booleanity Check

The *Booleanity Check* takes a column $c = (d, a)$ and proves that every active entry of c is 0 or 1. We package this as the relation

$$\mathcal{R}_{\text{Bool}} = \{ \mathbf{x} = \perp, \mathbf{y} = (\llbracket \tilde{c} \rrbracket), \mathbf{w} = (c) \mid \forall i, a[i] = 1 \implies d[i] \in \{0, 1\} \}.$$

Since $d[i] \in \{0, 1\}$ exactly when $d[i](1-d[i]) = 0$, the check is a single Zerocheck on the column $a \cdot d \cdot (1-d)$, which vanishes everywhere iff every active entry is boolean.

PIOP 14: Booleanity Check

Inputs: $\llbracket \tilde{c} \rrbracket = (\llbracket \tilde{a} \rrbracket, \llbracket \tilde{d} \rrbracket)$.

1. **P** and **V** invoke Zerocheck on the column $a \cdot d \cdot (1-d)$.

B.3 No-Duplicate Check

The *No-Duplicate Check* takes a column $c = (d, a)$ and proves that the active entries of c are pairwise distinct. We package this as the relation

$$\mathcal{R}_{\text{NoDup}} = \{ \mathbf{x} = \perp, \mathbf{y} = (\llbracket \tilde{c} \rrbracket), \mathbf{w} = (c) \mid \forall i \neq j, a[i] = a[j] = 1 \implies d[i] \neq d[j] \}.$$

PIOP 15: No-Duplicate Check

TODO

B.4 Limb Reconstruction Check

The *Limb Reconstruction Check* takes 1 n -bit column c and k columns $c_0, \dots, c_{k-1}$ each with n/k bits and proves that each entry of c is the concatenation of the corresponding entries in $c_0, \dots, c_{k-1}$. We package this as the relation

$$\mathcal{R}_{\text{Limb}} = \left\{ \mathbf{x} = n, \mathbf{y} = (\llbracket \tilde{c} \rrbracket, \llbracket \tilde{c}_0 \rrbracket, \dots, \llbracket \tilde{c}_{k-1} \rrbracket), \mathbf{w} = (c, c_0, \dots, c_{k-1}) \mid \forall i, c[i] = \sum_{j=0}^{k-1} 2^{(k-1-j)n/k} c_j[i] \right\}.$$

Reconstruction is a single linear identity over the hypercube: subtracting the weighted limbs from c must vanish at every row. A single Zerocheck on $c - \sum_{j=0}^{k-1} 2^{(k-1-j)n/k} \cdot c_j$ enforces exactly this, pinning down each entry of c as the concatenation of the corresponding limbs. The check certifies only the weighted-sum relationship; bounding each limb c_j to its n/k -bit width, when required, is done by a separate Range Check.

PIOP 16: Limb Reconstruction Check

Inputs: $\llbracket \tilde{c} \rrbracket, \llbracket \tilde{c}_0 \rrbracket, \dots, \llbracket \tilde{c}_{k-1} \rrbracket$.

1. **P** and **V** invoke Zerocheck on the column $c - \sum_{j=0}^{k-1} 2^{(k-1-j)n/k} \cdot c_j$.

B.5 Lookup Check

The *Lookup Check* takes two columns $c_1 = (d_1, a_1)$ and $c_2 = (d_2, a_2)$ and proves that every active value of c_1 also occurs as an active value of c_2 , written $c_1 \sqsubseteq c_2$. We package this as the relation

$$\mathcal{R}_{\text{Lookup}} = \{ \mathbf{x} = \perp, \mathbf{y} = (\llbracket \tilde{c}_1 \rrbracket, \llbracket \tilde{c}_2 \rrbracket), \mathbf{w} = (c_1, c_2) \mid \forall i, a_1[i] = 1 \implies \exists j : a_2[j] = 1 \wedge d_1[i] = d_2[j] \}.$$

We discharge the lookup with the LogUp technique [Hab22]: $c_1 \sqsubseteq c_2$ holds if and only if there is a *multiplicity* column m over c_2 's hypercube such that, with μ and ν the hypercube dimensions of $\tilde{c}_1$ and $\tilde{c}_2$,

$$\sum_{x \in \mathcal{H}_\mu} \frac{\tilde{a}_1(x)}{X - \tilde{d}_1(x)} = \sum_{y \in \mathcal{H}_\nu} \frac{\tilde{a}_2(y) \cdot \tilde{m}(y)}{X - \tilde{d}_2(y)} \quad (1)$$

as rational functions in X , where the activators zero out the inactive terms on each side. The prover sends m , and the parties verify the rational identity by Schwartz–Zippel: the verifier samples a random $r \xleftarrow{\$} \mathbb{F}$, the prover sends the purported common value s that both sides take at $X = r$, and the parties run RatSumcheck twice to confirm that each side evaluates to s at $X = r$. When c_2 is known to be duplicate-free, $m(y)$ records the multiplicity of $\tilde{d}_2(y)$ in c_1 , so the verifier stores $\llbracket \tilde{m} \rrbracket$ as a protocol output for use by higher-level checks.

PIOP 17: Lookup Check

Inputs: $\llbracket \tilde{c}_1 \rrbracket, \llbracket \tilde{c}_2 \rrbracket$.

1. **P** constructs and sends the multiplicity oracle $\llbracket \tilde{m} \rrbracket$, which **V** stores as protocol output.
2. **V** samples a random point $r \xleftarrow{\$} \mathbb{F}$ and sends it to **P**, who responds with the claimed common value s of both sides of Eq. (1) at $X = r$.
3. **P** and **V** invoke RatSumcheck twice to prove that both sides of Eq. (1) evaluate to s at $X = r$.

B.6 Range Check

The *Range Check* takes a column $c = (d, a)$ and a width n , and proves that every active entry of c is a non-negative integer of bitwidth n , i.e. lies in $[0, 2^n - 1]$. We package this as the relation

$$\mathcal{R}_{\text{Range}} = \{ \mathbf{x} = n, \mathbf{y} = (\llbracket \tilde{c} \rrbracket), \mathbf{w} = (c) \mid \forall i, a[i] = 1 \implies 0 \leq d[i] < 2^n \}.$$

We assume $n < \log_2 |\mathbb{F}|$, so the range embeds unambiguously into $\mathbb{F}$. A direct approach proves $c \sqsubseteq \text{rng}_n := (1, \mathcal{I}_n)$, since the index polynomial $\mathcal{I}_n$ ranges over exactly $\{0, \dots, 2^n - 1\}$; but this lookup table has 2^n rows and is wasteful for large n . Instead we split each value into 16-bit *limbs* and range-check each limb against a small index column. Choosing k and β with $n = 16k + \beta$ and $\beta \leq 16$, the prover sends limb columns $d_1, \dots, d_{k+1}$ satisfying

$$a(x) \cdot d(x) = \sum_{i=1}^{k+1} 2^{16(i-1)} \cdot d_i(x), \quad x \in \mathcal{H}_\mu,$$

which a single Zerocheck enforces; the activator collapses inactive rows to zero, so the identity pins the limb decomposition on active rows. A Lookup Check of each limb into the appropriate small index column then forces $d(x) \in [0, 2^n - 1]$ at every active x .

PIOP 18: Range Check

Inputs: $[\tilde{c}] = ([\tilde{a}], [\tilde{d}])$, width n .

1. **P** and **V** compute β, k such that $16k + \beta = n$ and $\beta \leq 16$.
2. **P** constructs and sends the limb oracles $[\tilde{d}_1], \dots, [\tilde{d}_{k+1}]$, the 16-bit limb decomposition of $\tilde{d}$.
3. **P** and **V** invoke Zerocheck on the column $\tilde{a} \cdot \tilde{d} - \sum_{i=1}^{k+1} 2^{16(i-1)} \cdot \tilde{d}_i$.
4. For each $i \in [k]$, **P** and **V** invoke Lookup Check on $(1, \tilde{d}_i)$ and $\text{rng}_{16} := (1, \mathcal{I}_{16})$, proving each limb lies in $[0, 2^{16} - 1]$.
5. **P** and **V** invoke Lookup Check on $(1, \tilde{d}_{k+1})$ and $\text{rng}_\beta := (1, \mathcal{I}_\beta)$, proving the residual limb lies in $[0, 2^\beta - 1]$.

B.7 Sign Check

The *Sign Check* takes a column $c = (d, a)$, a width α , and a sign mode $\sigma \in \{\text{NonNeg}, \text{Pos}, \text{NonPos}, \text{Neg}\}$, and proves that every active entry of c lies in the width- α slice of the corresponding sign. Each mode is determined by two parameters: a *direction* $s_\sigma \in \{+1, -1\}$, which is $+1$ for the positive-side modes NonNeg, Pos and -1 for the negative-side modes NonPos, Neg; and a lower bound $\ell_\sigma \in \{0, 1\}$, which is 0 for the non-strict modes NonNeg, NonPos and 1 for the strict modes Pos, Neg that exclude zero. Concretely,

σ	NonNeg	Pos	NonPos	Neg
certified range	$[0, 2^\alpha - 1]$	$[1, 2^\alpha - 1]$	$[-(2^\alpha - 1), 0]$	$[-(2^\alpha - 1), -1]$
(s_σ, ℓ_σ)	$(+1, 0)$	$(+1, 1)$	$(-1, 0)$	$(-1, 1)$

where negative values are taken in $\mathbb{F}$ via the additive inverse. In every mode the certified condition is $s_\sigma \cdot d[i] \in [\ell_\sigma, 2^\alpha - 1]$, so we package the check as the relation

$$\mathcal{R}_{\text{Sign}} = \{ \mathbf{x} = (n, \alpha, \sigma), \mathbf{y} = ([\tilde{c}]), \mathbf{w} = (c) \mid \forall i, a[i] = 1 \implies \ell_\sigma \leq s_\sigma \cdot d[i] \leq 2^\alpha - 1 \}.$$

We assume $\alpha < \log_2 |\mathbb{F}|$, so the range embeds unambiguously into $\mathbb{F}$.

The whole check reduces to a non-negativity check on the *magnitude* column $m := s_\sigma \cdot d$, which the verifier obtains by negating the oracle $\tilde{d}$ in the negative-side modes. The index polynomial $\mathcal{I}_\alpha$, evaluated over $\mathcal{H}_\alpha$, takes exactly the values $\{0, 1, \dots, 2^\alpha - 1\}$. Treating it as the data of an α -variable column with the all-ones activator gives a range column $\text{rng}_\alpha := (1, \mathcal{I}_\alpha)$ whose value multiset is precisely $[0, 2^\alpha - 1]$, so a single Lookup Check of m into rng_α is already an exact range check. This naive lookup is wasteful when α is large, since rng_α has 2^α rows. We do better by splitting each entry into 16-bit *limbs* and range-checking each limb against a small index column. Choosing k and β with $\alpha = 16k + \beta$ and $\beta \leq 16$, the prover sends limb columns $m_1, \dots, m_{k+1}$ satisfying

$$a(x) \cdot s_\sigma \cdot d(x) = \sum_{i=1}^{k+1} 2^{16(i-1)} \cdot m_i(x), \quad x \in \mathcal{H}_\mu,$$

which a single Zerocheck enforces; the activator on the left collapses inactive rows to zero, so the identity pins down the limb decomposition on active rows and forces the limbs to combine to zero on inactive rows. Range-checking each limb then pins $s_\sigma \cdot d(x) \in [0, 2^\alpha - 1]$ at every active x . In the strict modes Pos and Neg, a final no-zero check rules out the value 0, narrowing the certified range to $[1, 2^\alpha - 1]$.

PIOP 19: Sign Check

Inputs: $\llbracket \tilde{c} \rrbracket = (\llbracket \tilde{a} \rrbracket, \llbracket \tilde{d} \rrbracket)$, width α , sign mode $\sigma \in \{\text{NonNeg}, \text{Pos}, \text{NonPos}, \text{Neg}\}$.

1. **P** and **V** read off $s_\sigma \in \{+1, -1\}$ and $\ell_\sigma \in \{0, 1\}$ from σ , and compute β, k such that $16k + \beta = \alpha$ and $\beta \leq 16$.
2. **P** constructs and sends the limb oracles $\llbracket \tilde{m}_1 \rrbracket, \dots, \llbracket \tilde{m}_{k+1} \rrbracket$ of the magnitude $m = s_\sigma \cdot d$.
3. **P** and **V** invoke Zerocheck to prove $a \cdot s_\sigma \cdot d - \sum_{i=1}^{k+1} 2^{16(i-1)} \cdot m_i = 0$.
4. For each $i \in [k]$, **P** and **V** invoke Lookup Check on $(1, m_i)$ and $\text{rng}_{16} := (1, \mathcal{I}_{16})$, proving each limb lies in $[0, 2^{16} - 1]$.
5. **P** and **V** invoke Lookup Check on $(1, m_{k+1})$ and $\text{rng}_\beta := (1, \mathcal{I}_\beta)$, proving the residual limb lies in $[0, 2^\beta - 1]$.
6. If $\ell_\sigma = 1$ (the strict modes Pos, Neg), **P** and **V** additionally invoke NoZero Check on c to rule out zero entries, narrowing the certified range to $[1, 2^\alpha - 1]$.

B.8 Keyed Sumcheck

The *Keyed Sumcheck* takes two tables L and R , each with a key column key , a value column val , and an activator column a , and proves that for every possible key $\kappa \in \mathbb{F}$, the sum of active values in L with key κ equals the sum of active values in R with key κ . Define the keyed sums

$$S_L(\kappa) := \sum_{\substack{i \in [L] \\ L.\text{key}[i] = \kappa}} L.a[i] \cdot L.\text{val}[i] \quad \text{and} \quad S_R(\kappa) := \sum_{\substack{j \in [R] \\ R.\text{key}[j] = \kappa}} R.a[j] \cdot R.\text{val}[j].$$

We package this as the relation

$$\mathcal{R}_{\text{ksum}} = \left\{ \mathbf{x} = \perp, \mathbf{y} = (\llbracket \tilde{L} \rrbracket, \llbracket \tilde{R} \rrbracket), \mathbf{w} = (L, R) \mid \forall \kappa \in \mathbb{F} : S_L(\kappa) = S_R(\kappa) \right\}.$$

KeyedSumcheck reduces the per-key equality $\forall \kappa : S_L(\kappa) = S_R(\kappa)$ to a single rational identity. The protocol is a generalization of the LogUp lookup protocol [Hab22], and we derive its security from a core lemma about function equality that is implicit in LogUp.

Lemma 1 (function equality [Hab22]). *For any functions $S_1, S_2 : \mathbb{F} \rightarrow \mathbb{F}$, $S_1(\kappa) = S_2(\kappa)$ for every $\kappa \in \mathbb{F}$ if and only if*

$$\sum_{\kappa \in \mathbb{F}} \frac{S_1(\kappa)}{X - \kappa} = \sum_{\kappa \in \mathbb{F}} \frac{S_2(\kappa)}{X - \kappa}$$

as rational functions in the formal variable X .

Applying Lemma 1 to S_L and S_R and collapsing the per-key double sums into single sums over rows (the per-key index sets partition the rows), the property holds if and only if, with μ and ν the hypercube dimensions of $\tilde{L}$ and $\tilde{R}$,

$$\sum_{x \in \mathcal{H}_\mu} \frac{\tilde{L}.a(x) \cdot \tilde{L}.\text{val}(x)}{X - \tilde{L}.\text{key}(x)} = \sum_{x \in \mathcal{H}_\nu} \frac{\tilde{R}.a(x) \cdot \tilde{R}.\text{val}(x)}{X - \tilde{R}.\text{key}(x)}. \quad (2)$$

We verify this rational identity by Schwartz–Zippel: the verifier samples a random $r \xleftarrow{\$} \mathbb{F}$, the prover sends the purported common value s that both sides take at $X = r$, and the parties run RatSumcheck twice to confirm that both sides actually evaluate to s at $X = r$.

PIOP 20: KeyedSumcheck

Inputs: $\llbracket \tilde{L} \rrbracket, \llbracket \tilde{R} \rrbracket$.

1. **V** samples a random point $r \xleftarrow{\$} \mathbb{F}$ and sends it to **P**.
2. **P** computes and sends **V** the value s , the purported evaluation of both sides of Eq. (2) at $X = r$.
3. **P** and **V** invoke RatSumcheck twice to prove that both sides of Eq. (2) evaluate to s at $X = r$.

B.9 Multiset Equality Check

The *Multiset Equality Check* takes two columns c_L and c_R and proves that they hold the same multiset of active values. We package this as the relation

$$\mathcal{R}_{\text{mset}} = \{ \mathbb{x} = \perp, \mathbb{y} = (\llbracket \tilde{c}_L \rrbracket, \llbracket \tilde{c}_R \rrbracket), \mathbb{w} = (c_L, c_R) \mid \{ \{ c_L[i] \} \}_i = \{ \{ c_R[j] \} \}_j \}.$$

Two multisets agree exactly when every value $\kappa \in \mathbb{F}$ occurs with the same multiplicity on both sides, i.e. $\forall \kappa \in \mathbb{F} : \sum_{i: c_L[i]=\kappa} 1 = \sum_{j: c_R[j]=\kappa} 1$. This is precisely an κ -keyed sum equality between two tables whose key is the column data and whose value is the constant 1. The parties locally form the auxiliary tables

$$L := (a = c_L.a, \text{key} = c_L.data, \text{val} = 1) \quad \text{and} \quad R := (a = c_R.a, \text{key} = c_R.data, \text{val} = 1),$$

and a single KeyedSumcheck on L and R closes out the proof, with completeness and soundness inherited from the keyed sumcheck.

PIOP 21: Multiset Equality Check

Inputs: $\llbracket \tilde{c}_L \rrbracket, \llbracket \tilde{c}_R \rrbracket$.

1. **P** and **V** locally form the tables L and R as above.
2. **P** and **V** invoke KeyedSumcheck on L and R .

Handling tables. The foregoing column protocol extends to prove that two *tables* hold the same multiset of *rows*: the parties first fingerprint each table to a single column with a verifier-chosen random linear combination of its data columns, and then invoke the foregoing check on the two fingerprint columns.

B.10 Rotation Check

The *Rotation Check* takes two columns c and c' together with a shift $t \in \mathbb{Z}$, and proves that c' is the cyclic rotation of c by t positions, written $c' = \rho_t(c)$, where ρ_t sends the entry at index p to index $p + t$ modulo the column length s . We package this as the relation

$$\mathcal{R}_{\text{rot}} = \left\{ \mathbb{x} = t, \mathbb{y} = (\llbracket \tilde{c} \rrbracket, \llbracket \tilde{c}' \rrbracket), \mathbb{w} = (c, c') \mid \begin{array}{l} |c| = |c'| =: s \\ \forall q \in [s] : c'[q] = c[(q - t) \bmod s] \end{array} \right\}.$$

We first adopt the contiguous convention so that the active rows of both columns occupy hypercube indices $0, \dots, s - 1$, which the verifier enforces by asserting $c.a = c'.a = C_{\mu, s}$. We then reduce the rotation condition to a single multiset equality on *position-tagged* columns: tag each entry of c with its own position, and tag each entry of c' with the position it should occupy in c . Concretely, the parties locally construct the tagged tables

$$T := (c.a, c.data, \mathcal{I}_\mu) \quad \text{and} \quad T' := (c'.a, c'.data, \mathcal{I}_\mu^{\text{rot}}),$$

where $\mathcal{I}_\mu$ maps each $x \in \mathcal{H}_\mu$ to its hypercube position and $\mathcal{I}_\mu^{\text{rot}}$ is the *shifted* index polynomial sending each active position q to $(q - t) \bmod s$. Writing $t_0 := t \bmod s$, this shift is realized exactly by

$$\mathcal{I}_\mu^{\text{rot}}(x) := \mathcal{I}_\mu(x) - t_0 + s \cdot \mathcal{C}_{\mu, t_0}(x),$$

since $\mathcal{C}_{\mu, t_0}(x) = 1$ precisely when $\mathcal{I}_\mu(x) < t_0$, supplying the $+s$ wraparound exactly on the positions that underflow. Because $q \mapsto (q - t) \bmod s$ is a bijection on $\{0, \dots, s - 1\}$, the tagged tables agree as multisets of rows if and only if $c'[q] = c[(q - t) \bmod s]$ for every q , which is exactly the rotation condition. A single Multiset Equality Check on T and T' then proves the claim, with completeness and soundness inherited from the multiset equality check.

PIOP 22: Rotation Check

Inputs: $\llbracket \tilde{c} \rrbracket, \llbracket \tilde{c}' \rrbracket, t$.

1. **P** sends **V** the active length s , and **V** asserts $c.a = c'.a = \mathcal{C}_{\mu, s}$.
2. **P** and **V** locally construct the tagged tables T and T' as above, using the shifted index polynomial $\mathcal{I}_\mu^{\text{rot}}(x) = \mathcal{I}_\mu(x) - t_0 + s \cdot \mathcal{C}_{\mu, t_0}(x)$ with $t_0 := t \bmod s$.
3. **P** and **V** invoke Multiset Equality Check to prove that T and T' are equal as multisets of rows.

References

- [BCGGRS19] E. Ben-Sasson, A. Chiesa, L. Goldberg, T. Gur, M. Riabzev, and N. Spooner. “Linear-Size Constant-Query IOPs for Delegating Computation”. In: *Proceedings of the 17th Theory of Cryptography Conference*. TCC ’19, 2019.
- [BFS20] B. Bünz, B. Fisch, and A. Szepieniec. “Transparent SNARKs from DARK Compilers”. In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’20, 2020, pp. 677–706.
- [BMMS25] A. Baweja, P. Mishra, T. Mopuri, and M. Shtepel. “FICS and FACS: Fast IOPPs and Accumulation via Code-Switching”. Cryptology ePrint Archive, Paper 2025/737. 2025.
- [BMNW25] B. Bünz, P. Mishra, W. Nguyen, and W. Wang. “Arc: Accumulation for Reed–Solomon Codes”. In: *Proceedings of the 45th Annual International Cryptology Conference*. CRYPTO ’25, 2025.
- [CHMMVW20] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward. “Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS”. In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’20, 2020.
- [GWC19] A. Gabizon, Z. J. Williamson, and O. Ciobotaru. “PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge”. Cryptology ePrint Archive, Report 2019/953, 2019.
- [Hab22] U. Haböck. “Multivariate lookups based on logarithmic derivatives”. Cryptology ePrint Archive, Paper 2022/1530. <https://eprint.iacr.org/2022/1530>. 2022.
- [KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. “Constant-Size Commitments to Polynomials and Their Applications”. In: *Proceedings of the 16th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’10, 2010, pp. 177–194.
- [Lam+24] A. Lamb, Y. Shen, D. Heres, J. Chakraborty, M. O. Kabak, L.-C. Hsieh, and C. Sun. “Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine”. In: *Companion of the 2024 International Conference on Management of Data*. SIGMOD ’24. Association for Computing Machinery, 2024, pp. 5–17. DOI: 10.1145/3626246.3653368.
- [NSSMM26] B. Namboothiry, A. Shirzad, S. Solit, R. Marcus, and P. Mishra. “TruthTable: A Verifiable Query Engine”. Cryptology ePrint Archive, Paper 2026/1260. 2026.
- [PST13] C. Papamanthou, E. Shi, and R. Tamassia. “Signatures of Correct Computation”. In: *Proceedings of the 10th Theory of Cryptography Conference*. TCC ’13, 2013, pp. 222–242.
- [STZRK25] M. Stoian, J. Thürauf, A. Zimmerer, A. van Renen, and A. Kipf. “Instance-Optimized String Fingerprints”. In: *Proceedings of Workshops at the 51st International Conference on Very Large Data Bases, VLDB 2025, London, UK, September 1-5, 2025*. Ed. by N. W. Paton and J. Paparrizos. VLDB.org, 2025.